

Throwback Dynamics with Repeated Entries

Recurrence classification, exact cycles,
perfect mixing, and extremal periods

19 September 2026

Abstract

A throwback move removes the first entry of an infinite queue and reinserts it at the zero-based position equal to its positive integer value. We resolve the repeated-entry recurrence question in an occurrence-labeled formulation, and also identify exactly which numerical values recur. Every occurrence is recurrent precisely when, for every m , at most m entries have value at most m . When this condition fails, the first overloaded initial prefix identifies the complete recurrent core; the remaining tail never participates. A sparse backward-parking algorithm extracts this core without simulating the queue. We then determine every cycle length of the bounded configuration system, obtain an explicit coprimality criterion for perfect mixing, and prove that the largest labeled period of a recurrent core of size r is $\max_{2 \leq k \leq r} k^{r-k+1}$. This gives a throwback interpretation of OEIS A003320. Catalan and prime-parking-function enumerations, rational generating functions, a Lambert- W asymptotic, and an exact example of escaped limiting density complete the analysis. All main statements have proofs; executable standard-library Python checks accompany the manuscript.

Scope and status. The starting questions are explicitly raised by Darnell and Dribus in their 2025 paper. This manuscript presents proposed resolutions with self-contained proofs, not a claim of established publication priority. Literature checking did not locate a later resolution, but was not exhaustive. The proofs have not been independently refereed or formalized in a proof assistant. Computation is used as an additional check, not as a substitute for proof.

Contents

1	The problem and its provenance	1
1.1	What the source check establishes	1
2	Labeled queues and closed fronts	1
3	A quantitative waiting-or-barrier lemma	2
4	The first overloaded prefix determines everything	3
4.1	Why the first prefix, rather than the global smallest value, matters	4
5	Examples, including a nonlocal-looking obstruction	4
6	A sparse algorithm that avoids dynamical simulation	5
6.1	Backward parking	5
6.2	Recovering the least threshold without sorting	5
7	Counting all possible recurrent cores	7
7.1	A cyclic-minimum lemma	7
7.2	Generating functions and asymptotics	8
8	Finite bounded configurations and exact cycles	8
8.1	The autonomous state space	8
8.2	Deleting a minimum-weight clock	9
9	A complete coprimality test for perfect mixing	10
10	Infinite frequencies, repetitions, and escaped density	11
10.1	Equal weights have equal individual frequencies	11
10.2	When do these frequencies sum to one?	12
10.3	Uniform recurrence for nondecreasing inputs	12
11	The largest possible recurrent-core period	13
11.1	An exact two-candidate rule and a Lambert- W asymptotic	14
12	Rational generating functions and exact simulation	15
12.1	A finite exact model from a prefix certificate	15
12.2	Avoiding a truncation error	16
13	Computational verification and reproducibility	16
13.1	Every small closed input, with repeated weights distinguished	16
13.2	Complete bounded configuration spaces	16
13.3	Additional independent checks	17
13.4	Reproduction commands	17
14	Scope, remaining refinements, and conclusion	17
A	A compact proof-audit checklist	18
B	Code and data inventory	19

1 The problem and its provenance

Darnell and Dribus prove recurrence for distinct positive inputs and leave the repeated-entry case unresolved immediately after their Theorem 2 [1, p. 4]. They also ask for a characterization of perfect mixing [1, p. 16]. These are the two questions addressed here. Their frequency and placement results for distinct inputs provide useful context, but we give complete proofs of the extensions used below.

The motivating OEIS entry A357081 records the leader sequence obtained from $(3, 4, 5, \dots)$ [2]. Our target is not its original recurrence conjecture, which is already covered by the 2025 theorem. Instead, the central issue is the effect of repetitions: equal values can form an active front while all later entries become inaccessible.

A necessary distinction is easy to overlook. In a queue consisting entirely of 1's, the *numerical value* 1 recurs forever, but only the first two *occurrences* ever lead. We therefore retain occurrence labels throughout. The resulting theorem determines numerical recurrence too, without confusing these two meanings.

The principal contributions are the first-obstruction theorem (Theorem 4.1), its algorithmic implementation (Theorem 6.2), the exact cycle theorem (Theorem 8.3), and the extremal-period theorem (Theorem 11.1). The perfect-mixing criterion is Corollary 9.1. The familiar sequences that appear in the enumeration and extremal formula are not claimed to be new sequences.

1.1 What the source check establishes

The questions were checked against the actual 2025 article, including the relevant PDF pages, rather than inferred from an OEIS “conjecture” tag. Public searches and the relevant OEIS pages were consulted on 19 September 2026. No later treatment of the two selected questions was located. This negative search result is a limitation of the literature check, not a proof that no earlier or unpublished solution exists. The mathematical content below is intended to be checked independently of that priority question.

2 Labeled queues and closed fronts

Let $X = (x_0, x_1, \dots)$ be an infinite sequence with $x_i \in \mathbb{Z}_{>0}$. Token i has permanent weight x_i . A queue state is an ordering of these tokens; its positions are numbered $0, 1, 2, \dots$. If the current head is token i , one move removes it and reinserts it at position x_i . Thus the numerical operation is

$$\tau(x_0, x_1, \dots) = (x_1, \dots, x_{x_0}, x_0, x_{x_0+1}, \dots). \quad (1)$$

Let Q_t denote the state before move t , let L_t be its head token, and let $T_t = x_{L_t}$. The sequences (L_t) and (T_t) are respectively the *labeled* and *numerical* leader words.

A token is *recurrent* when it occurs infinitely often in (L_t) . A numerical value is recurrent when it occurs infinitely often in (T_t) . The distinction matters whenever weights repeat.

Definition 2.1. A finite multiset A of positive weights is *admissible* if

$$C_A(m) := \#\{a \in A : a \leq m\} \leq m \quad (m \geq 1). \quad (2)$$

An infinite input is admissible when the same inequalities hold with all its occurrences counted. Counts may initially be infinite; an admissible input necessarily has finite multiplicity below every fixed bound.

If the sorted weights of a finite multiset are $a_0 \leq \dots \leq a_{n-1}$, then admissibility is equivalent to

$$a_i \geq i + 1 \quad (0 \leq i < n). \quad (3)$$

Indeed, $a_i \leq i$ exhibits at least $i + 1$ weights at most i . Conversely, $C_A(m) > m$ forces $a_m \leq m$. Admissibility is inherited by submultisets.

Definition 2.2. A *closed front of size r* is a state whose first r tokens all have weight at most $r - 1$. A multiset of $r \geq 2$ positive weights is *critical* if

$$\max A = r - 1, \quad C_A(m) \leq m \quad (1 \leq m \leq r - 2). \quad (4)$$

A closed front stays closed: every subsequent move is wholly inside those r positions. Tokens behind it remain at exactly the same positions, not merely at positions with a limiting distribution.

Two elementary observations will be used repeatedly. A token not currently leading can only stay put or move one position left. Once token i has been thrown, its position is always at most x_i . Also, the relative order of never-yet-leading tokens never changes. In particular, first appearances as leader occur in original label order $0, 1, 2, \dots$, possibly stopping at a finite label.

3 A quantitative waiting-or-barrier lemma

The following lemma is useful because it does not assume admissibility. Its obstruction is an actual closed front, rather than an unspecified failure of recurrence.

Lemma 3.1 (Waiting or a barrier). *Suppose a token A is currently at position p . Within $2^p - 1$ moves, one of the following has happened: A has been a leader, or a closed front of size at most p , lying strictly before A , has formed. In the second case A has not yet led during this waiting interval and can never lead afterward.*

Proof. Induct on p . At $p = 0$ the token is already the leader. Suppose $p > 0$. If all p tokens before A have weight at most $p - 1$, they form the required closed front at once. Otherwise choose a token B before A whose weight is at least p . Its position is at most $p - 1$.

Apply the induction hypothesis to B . A barrier before B is also before A , since before either token first leads their relative order cannot change. Alternatively, B leads after at most $2^{p-1} - 1$ moves. At that time A has position at most p . Throwing B shifts A at least one position left, because B 's weight is at least p . Now A is at position at most $p - 1$. A second application of induction needs at most another $2^{p-1} - 1$ moves. The total is at most

$$(2^{p-1} - 1) + 1 + (2^{p-1} - 1) = 2^p - 1.$$

Every barrier supplied by this argument excludes A and is permanent. □

Corollary 3.2. *An admissible infinite input has no closed front at any time. Every token initially at position j first leads by time $2^j - 1$. A recurrent visit by a token of weight v is followed by another within at most 2^v moves.*

Proof. A closed front of size r would contain r weights at most $r - 1$, contrary to admissibility. Apply Lemma 3.1 initially, and then after each throw, when the token has position v . The extra move for the throw changes the bound $2^v - 1$ to 2^v between leader times. $\square$

The numerical bound is deliberately simple, not generally optimal. Its importance here is that it gives finite-time conclusions with no probabilistic or compactness argument.

4 The first overloaded prefix determines everything

For $j \geq 0$, let A_j be the multiset of weights in the original prefix $(x_0, \dots, x_j)$. Write $A_{-1} = \emptyset$.

Theorem 4.1 (First-obstruction classification). *Exactly one of the following alternatives holds.*

Admissible alternative. *If*

$$\boxed{\#\{i \geq 0 : x_i \leq m\} \leq m \quad \text{for every } m \geq 1,} \quad (5)$$

then every labeled token leads infinitely often.

Obstructed alternative. *Otherwise let*

$$J = \min\{j \geq 0 : A_j \text{ is not admissible}\}, \quad (6)$$

$$m = \min\{k \geq 1 : C_{A_J}(k) > k\}. \quad (7)$$

Then $x_J \leq m$, $C_{A_J}(m) = m + 1$, and the recurrent token set is exactly

$$\boxed{\mathcal{R} = \{i \leq J : x_i \leq m\}.} \quad (8)$$

Precisely the labels $0, 1, \dots, J$ ever lead. By the first appearance of label J , and in particular by time $2^J - 1$, the tokens in $\mathcal{R}$ occupy a closed front of size $m + 1$. Every position behind that front is then fixed. The multiset of recurrent weights is critical. These conclusions, and the entire leader word, are independent of the continuation after x_J .

Proof. If (5) holds, Corollary 3.2 proves recurrence. If it fails at a threshold k , finitely many occurrences already witness $k + 1$ entries of weight at most k . Thus a finite J in (6) exists.

The multiset A_{J-1} is admissible. Adding just one entry can increase any count by only one. Therefore the least violated threshold m satisfies

$$C_{A_J}(m) = m + 1, \quad C_{A_{J-1}}(m) = m, \quad x_J \leq m.$$

For each $k < m$, minimality of m gives $C_{A_J}(k) \leq k$. Consequently $\mathcal{R}$ has $m + 1$ tokens, all of weight at most m , and its counts below m satisfy the critical inequalities. At least one weight is m : otherwise $m + 1$ weights would already be at most $m - 1$. Thus its multiset is critical.

We next show that every label $j \leq J$ actually leads. Until its first appearance, every token in front of j has original label less than j : no later label can pass an unvisited token by becoming a leader first. A closed front strictly before j would therefore be a submultiset of A_{j-1} . But A_{j-1} is admissible, so such a front is impossible. Lemma 3.1, applied to the initial position j , proves that j first leads by time $2^j - 1$.

At the first leader time of J , every earlier token has already led and been thrown. The m earlier tokens in $\mathcal{R}$ therefore lie in positions at most m . Token J is at position zero and also belongs to $\mathcal{R}$.

These $m + 1$ distinct tokens fill all positions $0, \dots, m$, so the front is closed already at that time. No later original label has yet led, and none can lead afterward. This proves the assertion about all first appearances and the permanent tail.

It remains to check that every member of $\mathcal{R}$ is recurrent, rather than merely trapped in a larger closed front. A proper closed subfront of size $r < m + 1$ would require r core weights at most $r - 1$, violating criticality. Thus no proper barrier can occur inside the core. Repeated application of Lemma 3.1 gives recurrence of every core token.

Finally, the proof applies to every continuation of the prefix A_J . No token from that continuation ever leads, so its weight is never used to determine a move. Tracking the positions of labels $0, \dots, J$ therefore produces the same leader word for every continuation. $\square$

Corollary 4.2 (Sharp recurrence criterion). *For an infinite positive input, the following are equivalent: every token is recurrent; every token leads at least once; every finite initial prefix is admissible; and (5) holds.*

Proof. The only non-immediate implication is that every token leads at least once only in the admissible alternative. In the obstructed alternative, Theorem 4.1 says that no label larger than J ever leads. $\square$

Corollary 4.3 (Numerical recurrence). *In the admissible alternative every initial numerical value recurs. In the obstructed alternative a value v recurs if and only if $v = x_i$ for some $i \in \mathcal{R}$. Thus all initial numerical values recur exactly when the full input's set of values is contained in the core's set of values. In particular, for inputs with unbounded values, recurrence of every numerical value is equivalent to (5).*

Proof. In the obstructed alternative only finitely many noncore leader occurrences precede permanent capture. Each core token is recurrent. A finite core cannot represent an unbounded set of values. $\square$

4.1 Why the first prefix, rather than the global smallest value, matters

The recurrence criterion for *all* tokens is independent of their order. When recurrence fails, the identity of the surviving core need not be. For example,

$$(2, 2, 2, 1, 1, 6, 7, \dots) \quad \text{and} \quad (1, 1, 2, 2, 2, 6, 7, \dots)$$

have the same multiset. The first queue keeps its three initial 2-tokens active, while the second keeps its two initial 1-tokens active. Looking only for the globally least overloaded threshold would incorrectly predict the first queue's core. The prefix in (6) is essential.

5 Examples, including a nonlocal-looking obstruction

For

$$X = (2, 2, 3, 4, 5, \dots),$$

one has $C_X(1) = 0$ and $C_X(m) = m$ for $m \geq 2$. Repetition is harmless: every occurrence recurs. In contrast, a third initial 2 closes the first three positions immediately.

Consider the more instructive input

$$X = (2, 100, 2, 1, 1, \dots). \quad (9)$$

Its first three weights $(2, 100, 2)$ are admissible. Adding the first 1 creates the first bad prefix, with $J = 3$ and $m = 2$. Hence labels $0, 2, 3$ are exactly the recurrent core. Label 1, of weight 100, appears only transiently; label 4, though also of weight 1, never leads.

The initial labeled leaders are

$$0, 1, 2, 0, 3, 2, 3, 0, 3, 2, 3, 0, \dots$$

The numerical word is

$$2, 100, 2, 2, 1, 2, 1, 2, 1, 2, 1, 2, \dots$$

The eventual labeled period is 4, while the numerical period is 2. This difference is not cosmetic: cycle formulas in this manuscript always refer to labeled tokens unless explicitly stated otherwise.

For $X = (1, 2, 3, 1, \dots)$, the first bad prefix has $J = 3$, $m = 1$, and $\mathcal{R} = \{0, 3\}$. Thus the transiently encountered values 2 and 3 do not recur. For a constant input $(1, 1, 1, \dots)$ the same mechanism leaves only labels 0, 1, although its only numerical value is recurrent.

6 A sparse algorithm that avoids dynamical simulation

The inequalities defining admissibility are a nested matching condition. Think of each weight v as a deadline for a unit job that must occupy a distinct positive slot not exceeding v .

6.1 Backward parking

Process tokens in original order. Assign each new token of weight v the largest currently unoccupied slot at most v . Fail if no such slot exists. This is a static allocation procedure, not the throwback process itself.

Lemma 6.1. *Backward parking succeeds on a finite word if and only if its multiset is admissible. Its first failure occurs at exactly the index J in Theorem 4.1.*

Proof. Any successful allocation gives an injection of the tokens of weight at most m into slots $1, \dots, m$, proving necessity.

Suppose a token of weight v fails while all earlier tokens were parked. All slots $1, \dots, v$ are occupied. Let $M + 1$ be the first unoccupied slot larger than v . It exists because only finitely many slots have been used. Every slot $1, \dots, M$ is occupied. Every token parked in those slots has weight at most M : otherwise the still-free slot $M + 1$ would have been available when it parked, contradicting the choice of the largest free slot. Together with the failed token, there are therefore $M + 1$ weights at most M . Failure certifies inadmissibility. The two implications also identify the first failure. $\square$

6.2 Recovering the least threshold without sorting

A first failure identifies J , but the proof above may give a threshold larger than the least one. The following closure computation recovers the exact m in linear additional work.

Let $d(p)$ be the weight of the previously parked token occupying slot p . Start with $m = v = x_J$. Scan slots $p = 1, 2, \dots, m$ in order, extending the scan whenever necessary by replacing m with $\max(m, d(p))$. Stop when the scan reaches the current m . The scan never reaches an empty slot: the M from Lemma 6.1 bounds all extensions.

At termination, the first m slots are occupied by m tokens of weight at most m . Every earlier token of weight at most m must have parked in these slots, so $C_{A_{J-1}}(m) = m$. Conversely, if a threshold $k \geq v$ satisfies that equality, its first k slots are occupied precisely by weights at most k , and the closure cannot grow past k . Hence the terminal value is the least saturated threshold at least v , exactly the least threshold violated by adding v . The core is the set of occupants of slots $1, \dots, m$, together with the failed token.

Theorem 6.2 (Static extraction and certification). *Given a prefix through its first obstruction J , backward parking and the closure scan compute J , m , and the complete recurrent core. Using a union-by-size disjoint-set structure on occupied intervals, this takes $O((J+1)\alpha(J+1))$ amortized data-structure operations and $O(J+1)$ space. The bound uses unit-cost integer and dictionary operations; it is not a bit-complexity bound for arbitrarily large weights.*

Proof. The preceding lemmas prove correctness. The occupied slots form disjoint intervals. If v is free, it is the desired slot; otherwise let $[a, b]$ be the occupied component containing v . The desired slot is $a - 1$, with $a = 1$ meaning failure. Occupying one slot joins at most two neighboring components. Union by size, path compression, and a stored component minimum support these operations in the stated amortized bound. The final closure visits at most J occupied slots. $\square$

The distributed implementation uses sparse dictionaries keyed by occupied slots, so a weight such as 10^{40} does not allocate 10^{40} positions. The certificate has a deliberately independent verifier: it sorts the prefix before J and checks (3), then checks the least failure in the sorted extended prefix and the selected labels. A user need not trust the parking implementation to verify its output.

Listing 1: High-level static classification algorithm.

```

for each token j with weight v:
  p = largest free positive slot <= v
  if p exists:
    park token j in p
  else:
    m = v
    cursor = 1
    while cursor <= m:
      m = max(m, weight of occupant of slot cursor)
      cursor += 1
    return (J=j, threshold=m,
           core={occupants of slots 1,...,m} union {j})

```

On a finite admissible input, “no obstruction found” certifies only that inspected prefix. On an infinite admissible stream the search does not halt. This is an unavoidable distinction between finite evidence and an assertion about an arbitrary infinite continuation.

7 Counting all possible recurrent cores

A critical multiset is not merely necessary: any ordering of it, followed by any infinite tail, has that multiset as its complete recurrent core. Indeed, the initial front is closed and no proper closed subfront is possible. Thus critical multisets are the exact finite possibilities.

7.1 A cyclic-minimum lemma

Lemma 7.1. *Let $a_1, \dots, a_N$ be integers with sum 1. Exactly one of their N cyclic starting positions gives strictly positive partial sums.*

Proof. Put $S_0 = 0$ and $S_j = a_1 + \dots + a_j$. Start just after the last occurrence of the minimum of $S_0, \dots, S_{N-1}$. Unwrapped partial sums are positive by this last-minimum choice. Wrapped partial sums have the form $1 + S_j - S_k$ and are positive because the S_j are integers and $S_j \geq S_k$. Conversely, positivity of an unwrapped segment requires all later S_j to exceed S_k , while positivity after wrapping requires all earlier S_j to be at least S_k . Thus any successful start is precisely that last minimum. $\square$

Theorem 7.2 (Core enumeration). *For $r \geq 2$, the number of critical ordered weight vectors of length r is*

$$E_r = (r - 1)^{r-1}, \quad (10)$$

and the number of critical multisets is

$$H_r = \frac{1}{r} \binom{2r-2}{r-1} = C_{r-1}. \quad (11)$$

Consequently a uniformly chosen closed weight vector from $\{1, \dots, r-1\}^r$ has all r tokens recurrent with probability $1/(r-1)$.

Proof. Set $N = r - 1$ and replace each weight x by $b = r - x \in \{1, \dots, N\}$. If c_k is the number of these b 's equal to k , criticality becomes

$$c_1 + \dots + c_k > k \quad (1 \leq k \leq N), \quad c_1 + \dots + c_N = N + 1. \quad (12)$$

For example, the count in (12) is $r - C_A(r - k - 1)$, so the equivalence follows directly from (4).

Apply Lemma 7.1 to $c_k - 1$, whose sum is 1. Adding a common residue modulo N to all b -values cyclically rotates the counts. Every orbit of ordered words has size N , and exactly one word in it satisfies (12). There are N^{N+1} words, giving $N^{N+1}/N = N^N$ critical ordered vectors.

For multisets, count weak compositions of $N + 1$ into N parts. There are $\binom{2N}{N-1}$ such vectors. A count vector cannot have a nontrivial cyclic period: the number of repetitions would divide both N and $N + 1$. Thus all its cyclic orbits have size N , again with exactly one valid rotation. Their number is

$$\frac{1}{N} \binom{2N}{N-1} = \frac{1}{N+1} \binom{2N}{N}.$$

The probability statement follows by dividing (10) by $(r - 1)^r$. $\square$

The transformed objects in (12) are prime parking functions. Their known count is recorded in A000312; C_n is A000108 [4, 3]. The contribution here is their identification with throwback recurrent cores, not a new enumeration of parking functions or Catalan objects.

7.2 Generating functions and asymptotics

Let $C(z) = (1 - \sqrt{1 - 4z})/(2z)$. Formula (11) gives

$$\sum_{r \geq 2} H_r z^r = z(C(z) - 1) = \frac{1 - \sqrt{1 - 4z}}{2} - z. \quad (13)$$

In particular, $C_0 = 1$ and $C_n = \sum_{j=0}^{n-1} C_j C_{n-1-j}$ give a recurrence for H_r . Stirling's formula applied to (11) yields

$$H_r \sim \frac{4^{r-1}}{\sqrt{\pi} (r-1)^{3/2}}.$$

For the ordered count, let $U(z)$ be the formal solution of $U = ze^U$; equivalently, $U = -W(-z)$ on the branch analytic at zero. Then

$$\sum_{r \geq 2} E_r \frac{z^r}{r!} = 1 - e^{-U(z)} - z. \quad (14)$$

For completeness, Lagrange coefficient extraction gives

$$[z^r](1 - e^{-U(z)}) = \frac{1}{r} [u^{r-1}] e^{-u} e^{ru} = \frac{(r-1)^{r-1}}{r!}.$$

The $r = 1$ term is z , which is removed because a positive-weight core has at least two tokens. Thus the formal 0^0 convention used for that auxiliary coefficient does not introduce a size-one positive core.

8 Finite bounded configurations and exact cycles

The recurrence classification tells us which tokens survive. We now determine how the surviving or selected tokens move.

8.1 The autonomous state space

Select r labeled tokens whose weights, in nondecreasing order, are $s_0, \dots, s_{r-1}$. Assume either that this multiset is admissible and every unselected token has weight at least s_{r-1} , or that it is a critical closed core. Set

$$q_i = s_i - i + 1. \quad (15)$$

In the admissible case all $q_i \geq 2$. In the critical case all but the last are at least 2, and $q_{r-1} = 1$.

After each selected token has been thrown, its position p_i lies in $[0, s_i]$. The bounded state space is

$$\mathcal{C}(s) = \{(p_0, \dots, p_{r-1}) : 0 \leq p_i \leq s_i, p_i \neq p_j \ (i \neq j)\}. \quad (16)$$

Selecting positions in nondecreasing weight order proves

$$|\mathcal{C}(s)| = \prod_{i=0}^{r-1} (s_i + 1 - i) = \prod_{i=0}^{r-1} q_i. \quad (17)$$

All earlier chosen positions lie in the next token's allowed interval, so exactly i choices are excluded at stage i .

The dynamics on (16) is autonomous. If token h has position zero, its new position is s_h , and another position p_j decreases by one exactly when $p_j \leq s_h$. If no selected token is at zero, write the leader symbol as $*$. Its weight is at least every selected weight, so all selected positions decrease by one. In a closed core there is no $*$.

Lemma 8.1 (Explicit inverse). *The bounded-state map is a permutation of $\mathcal{C}(s)$. Given a state, find the selected tokens satisfying $p_i = s_i$. If none exists, its predecessor is $(p_i + 1)_i$. Otherwise choose among them a token h with least weight, put it at zero, increase by one every other position strictly less than s_h , and leave the larger positions unchanged.*

Proof. If no equality $p_i = s_i$ holds, every position may be increased by one without exceeding its bound; this is the inverse of a $*$ move. No selected throw could have produced the state, since that would leave the thrown token at its weight.

If h has the least weight among matching tokens, increasing a position $p_j < s_h$ cannot violate $p_j \leq s_j$. A violation would require $p_j = s_j < s_h$, giving a matching token of smaller weight. Thus the proposed predecessor is valid, and a throw of h recovers the given state.

No other predecessor is possible. A $*$ predecessor would exceed a matching token's bound. A throw from a larger matching weight would likewise push the smallest matching token beyond its bound when reversed. Matching tokens cannot tie in weight, since they would occupy the same position. This proves existence and uniqueness of the predecessor. In a closed core, a matching token always exists: the occupant of position $r - 1$ must have weight $r - 1$. $\square$

8.2 Deleting a minimum-weight clock

Lemma 8.2 (Minimum deletion). *In a bounded system, a selected token of minimum weight m leads once every $q = m + 1$ steps. After deleting that token and its leader times, the survivor process is the same bounded throwback process with all survivor weights reduced by one.*

Proof. Between its throws the minimum token has a position in $\{1, \dots, m\}$. Every other leader has weight at least m , so each intervening move shifts it left exactly one place. It therefore has constant return time $m + 1$.

When another token of weight v leads, the minimum token lies between positions 1 and m , and hence is crossed by that throw. Removing it reduces the throw distance from v to $v - 1$. A move of the minimum token changes no relative order among survivors and disappears in compressed time. At an instant when the minimum is the head, deleting it reduces every survivor position by one, placing the remaining selected tokens within their new bounds. The same reasoning applies to $*$ moves. $\square$

The closed recursion ends with a single token of reduced weight zero. Its word is constant and has period one. This is only a terminal bookkeeping system; the original queue still has strictly positive weights.

Theorem 8.3 (Exact periods, frequencies, and cycle counts). *For either bounded system above, define*

$$R_0 = 1, \quad R_{i+1} = R_i \frac{q_i - 1}{q_i}, \quad f_i = \frac{R_i}{q_i}. \quad (18)$$

Every cycle has the same least labeled state period $P(s)$. It is also the least period of the projected leader word (with a single symbol $*$ for unselected leaders). It is determined by

$$P_r = 1, \quad P_i = \frac{q_i P_{i+1}}{\gcd(P_{i+1}, q_i - 1)} \quad (i = r-1, r-2, \dots, 0), \quad P(s) = P_0. \quad (19)$$

The frequency of token i on every cycle is f_i , and the frequency of $*$ is R_r in the open case. Moreover,

$$P(s) = \text{lcm}_{0 \leq i < r} \text{den}(f_i), \quad \#\{\text{cycles}\} = \frac{\prod_i q_i}{P(s)}. \quad (20)$$

The denominator is taken after reducing each rational number.

Proof. Lemma 8.1 makes all bounded orbits periodic from the outset. Delete the minimum token using Lemma 8.2. By induction, suppose the reduced projected word has least period P' . Reinsert the minimum symbol at its fixed residue class modulo $q = q_0$. A period H of the full word must satisfy $q \mid H$, since the occurrence set of this unique labeled symbol is a single residue class modulo q . Write $H = q\ell$. The same shift advances the reduced word by $(q-1)\ell$ symbols. Hence it is a period exactly when

$$P' \mid (q-1)\ell.$$

The least such H is $qP' / \gcd(P', q-1)$. The empty open projection has the constant word $*$ and period one. The single zero-weight closed terminal system also has period one. Induction proves (19).

The minimum symbol occupies a proportion $1/q$ of the times. All survivor frequencies are multiplied by $(q-1)/q$. Iterating gives (18), including the residual $*$ frequency.

To verify that the leader word and the state have the same least period, extend the finite permutation orbit to all integer times. Every selected token occurs in the word, as its frequency above is positive. To determine its position at a time t , start just after any earlier occurrence of that token, where its position equals its weight, and replay the projected word. A $*$ move has a completely specified effect on its position. A period of the projected word thus gives the same position for every selected token after the shift. The state period divides the word period; the converse is immediate.

Finally, prove the denominator formula by the same induction. For an integer H to make all Hf_i integral, the first frequency $1/q$ requires $H = q\ell$. The survivor frequencies then require that $(q-1)\ell$ times every reduced frequency be integral, equivalently $P' \mid (q-1)\ell$ by induction. The residual frequency imposes no extra condition because it is one minus the sum of the selected frequencies. Thus the least denominator-clearing H is exactly the period already found. All states lie on equal-length cycles, so their number is the state count divided by that length. $\square$

9 A complete coprimality test for perfect mixing

Perfect mixing of a finite selected family means that its orbit visits every bounded configuration before returning. Theorem 8.3 turns this into an explicit arithmetic test.

Corollary 9.1 (Perfect-mixing criterion). *The bounded system with strides $q_0, \dots, q_{r-1}$ is a single cycle if and only if*

$$\boxed{\gcd(q_i - 1, q_j) = 1 \quad \text{for every } 0 \leq i < j < r.} \quad (21)$$

For an increasing infinite input with weights s_i , perfect mixing of every finite initial selected family is therefore equivalent to

$$\gcd(s_i - i, s_j - j + 1) = 1 \quad (i < j).$$

Proof. The period recursion has the form $P_0 = \prod_i q_i / \prod_i d_i$, where each d_i is the positive gcd appearing at its stage. Equality with the full state count requires every $d_i = 1$. Working backward, this is equivalent to

$$\gcd\left(q_i - 1, \prod_{j>i} q_j\right) = 1 \quad \text{for every } i,$$

which is exactly (21). For an increasing input, the initial positions i already satisfy the bounds, so there is no entry transient in the selected configuration system. $\square$

For consecutive weights beginning with a , all strides equal $a + 1$ and all gcds are $\gcd(a, a + 1) = 1$. This recovers the consecutive-input case of [1, Theorem 9].

Selected weights	Strides q_i	State count	Period	Cycles
(1, 3, 4)	(2, 3, 3)	18	18	1
(2, 3, 5)	(3, 3, 4)	36	9	4
(2, 4)	(3, 4)	12	6	2
(3, 6)	(4, 6)	24	8	3
(2, 2)	(3, 2)	6	3	2

For example, (2, 3, 5) fails because $\gcd(q_0 - 1, q_2) = \gcd(2, 4) = 2$. Its four cycles all have length nine. The criterion makes no claim that merely having pairwise coprime weights is sufficient; the relevant quantities are the shifted strides in (21).

10 Infinite frequencies, repetitions, and escaped density

Assume the infinite input is admissible. It has a well-defined sorted labeled list $s_0 \leq s_1 \leq \dots$, breaking equal-weight ties by original label. Every finite initial family eventually enters its bounded configuration space, because all its tokens eventually lead. Theorem 8.3 therefore proves, without exchanging an infinite sum and a limit, that the limiting frequency of sorted token i is

$$f_i = \frac{1}{s_i - i + 1} \prod_{j<i} \frac{s_j - j}{s_j - j + 1}. \quad (22)$$

For distinct weights this recovers the formula in [1, Theorem 4]; minimum deletion also justifies the repeated-weight case.

10.1 Equal weights have equal individual frequencies

Suppose a weight v occurs at sorted indices $a, \dots, a + b - 1$. Then $q_{a+t} = v - a - t + 1$. Telescoping gives

$$R_{a+t} = R_a \frac{v - a - t + 1}{v - a + 1}, \quad f_{a+t} = \frac{R_a}{v - a + 1} \quad (0 \leq t < b).$$

Thus tie-breaking does not affect individual frequencies. The numerical frequency of the value v is $bR_a/(v - a + 1)$. For $(2, 2, 3, 4, \dots)$ the individual frequencies begin

$$\frac{1}{3}, \frac{1}{3}, \frac{1}{6}, \frac{1}{12}, \frac{1}{24}, \dots$$

10.2 When do these frequencies sum to one?

Since $f_i = R_i - R_{i+1}$, finite telescoping gives

$$\sum_{i=0}^{n-1} f_i = 1 - R_n, \quad \sum_{i \geq 0} f_i = 1 - R_\infty, \quad R_\infty = \prod_{i \geq 0} \left(1 - \frac{1}{q_i}\right). \quad (23)$$

For $0 \leq u \leq 1/2$, $u \leq -\log(1 - u) \leq 2u$. Since $q_i \geq 2$, this proves

$$R_\infty = 0 \iff \sum_{i \geq 0} \frac{1}{q_i} = \infty. \quad (24)$$

Existence of every individual frequency alone does not imply that the sum is one. In particular, the general normalization sentence before Section 3 of [1, p. 3] needs this qualification; no infinite sum–limit interchange is justified merely by pointwise existence.

Here is an exact throwback example. Take

$$s_i = i + 2^{i+1} - 1 \quad (i \geq 0), \quad (25)$$

so $q_i = 2^{i+1}$. The input is strictly increasing and admissible, yet

$$R_\infty = \prod_{k \geq 1} (1 - 2^{-k}) = 0.288788095086602421278899721929 \dots,$$

$$\sum_i f_i = 1 - R_\infty = 0.711211904913397578721100278070 \dots$$

Every token has positive frequency and recurs, but a positive amount of density escapes to unbounded token ranks. There is no missing leader at any finite time; rather, density is not continuous under a decreasing countable intersection of these tail sets.

The displayed digits can be certified with rational arithmetic. If $R_N = \prod_{k=1}^N (1 - 2^{-k})$, the elementary inequality $\prod_k (1 - u_k) \geq 1 - \sum_k u_k$ gives

$$R_N(1 - 2^{-N}) \leq R_\infty \leq R_N. \quad (26)$$

The companion data use $N = 160$, making the interval width less than 2^{-160} .

10.3 Uniform recurrence for nondecreasing inputs

Proposition 10.1. *For every nondecreasing positive input, including repeated weights, every finite block of its labeled leader word reappears with bounded gaps.*

Proof. In the admissible case, the first r tokens start at positions $i \leq s_i$, so their bounded projected word is periodic from time zero. Any finite block of the full word uses only finitely many labels;

choose r containing them all. At that block, the projected word has no $*$ symbols. Repetition by its period reproduces the full block, with a fixed gap bound.

In the obstructed case, a nondecreasing input's first bad prefix is itself critical. Indeed, if its first failure occurs at index J , then $x_{J-1} \geq J$ when $J \geq 1$, whereas failure forces $x_J \leq J$; hence $x_J = J$ and all $J + 1$ initial weights are at most J . This prefix is closed and all its initial positions satisfy their weight bounds. Its labeled word is therefore periodic from time zero by Theorem 8.3. $\square$

For unsorted admissible inputs, finite small-weight projections are eventually periodic, but a finite transient block need not recur. The proposition is not silently asserting uniform recurrence for every arbitrary ordering.

11 The largest possible recurrent-core period

Let M_r be the largest *labeled* eventual period among critical cores of size $r \geq 2$. Although a core can have up to $r!$ possible labeled orders, its dynamics has a much sharper extremal period.

Theorem 11.1 (Sharp extremal period). *For every $r \geq 2$,*

$$M_r = \max_{2 \leq K \leq r} K^{r-K+1}. \quad (27)$$

For any candidate K , the critical profile

$$s_i = \min(K + i - 1, r - 1), \quad 0 \leq i < r, \quad (28)$$

has period exactly K^{r-K+1} .

Proof. Reverse the stride list of a critical profile, writing $a_t = q_{r-1-t}$ for $0 \leq t < r$. Criticality and sortedness imply

$$a_0 = 1, \quad a_t \geq 2 \ (t \geq 1), \quad a_{t+1} \leq a_t + 1.$$

Let p_t be the period accumulated through a_t in the backward recursion. Then $p_0 = 1$, each p_t is a multiple of a_t , and

$$p_{t+1} = \frac{a_{t+1}p_t}{\gcd(p_t, a_{t+1} - 1)}.$$

A unit rise from a to $a + 1$ is special: since $a \mid p_t$,

$$p_{t+1} = \frac{a+1}{a} p_t. \quad (29)$$

Thus consecutive unit rises telescope instead of multiplying their heights.

Partition the path $a_0, \dots, a_{r-1}$ into maximal runs of consecutive unit rises. The first run, ending at height h , changes the period from 1 to h . At the first step of any later run, from a height a to $b \leq a$, the period grows by at most the factor b . Subsequent unit rises in that run telescope, so the entire run grows the period by at most its final height h .

Let $K = \max_t a_t$, let A be the number of unit-rise edges, and let B be the number of runs. Then $B = r - A$, while reaching K from 1 requires $A \geq K - 1$. Every run ends at height at most K . Therefore

$$P(s) \leq K^B \leq K^{r-K+1}.$$

For equality at a prescribed K , use the reverse stride path

$$(1, 2, \dots, K, \underbrace{K, \dots, K}_{r-K \text{ further terms}}).$$

The first run has period K . Every later constant- K step multiplies the period by K , since a power of K is coprime to $K - 1$. Its period is K^{r-K+1} . Reversing the strides gives precisely (28), which is critical. Taking the maximum proves (27). $\square$

The values for $r = 2, 3, \dots$ begin

$$2, 4, 9, 27, 81, 256, 1024, 4096, 16384, 78125, 390625, 1953125, \dots$$

OEIS A003320 is defined by $a(n) = \max_{0 \leq k \leq n} k^{n-k}$ [5]. Thus

$$M_r = a(r+1) \quad (r \geq 2). \quad (30)$$

The extremal formula itself is a known integer sequence; its interpretation as the largest throwback core period is the result established here.

11.1 An exact two-candidate rule and a Lambert- W asymptotic

Write $N = r + 1$ and maximize

$$F_N(x) = (N - x) \log x.$$

Since

$$F'_N(x) = \frac{N}{x} - 1 - \log x, \quad F''_N(x) = -\frac{N}{x^2} - \frac{1}{x} < 0,$$

there is a unique continuous maximum. Let $w = W(eN)$ be the positive real solution of $we^w = eN$. The maximizing point is

$$x_* = \frac{N}{w} = e^{w-1}. \quad (31)$$

For $r \geq 3$, a maximizing integer is one of $\lfloor x_* \rfloor$ and $\lceil x_* \rceil$, so comparing those two exact integer powers evaluates M_r . The case $r = 2$ is $M_2 = 2$.

At the continuous optimum,

$$F_N(x_*) = N \left(w - 2 + \frac{1}{w} \right). \quad (32)$$

For sufficiently large N , rounding to the nearest integer changes x by at most $1/2$. Taylor's theorem and the displayed second derivative give

$$0 \leq N \left(w - 2 + \frac{1}{w} \right) - \log M_r = O \left(\frac{w^2}{N} \right). \quad (33)$$

In particular, the error here tends to zero, so this is also a relative asymptotic for M_r after exponentiating:

$$M_r = \exp \left[N \left(w - 2 + \frac{1}{w} \right) \right] \left(1 + O \left(\frac{(\log N)^2}{N} \right) \right). \quad (34)$$

The rounded value never exceeds the continuous one; the unsigned O notation in (34) does not discard that fact.

To make the scale more explicit, put $L = \log N$ and $\ell = \log L$. Solving $w + \log w = L + 1$ by substitution yields

$$w = L - \ell + 1 + \frac{\ell - 1}{L} + \frac{\ell^2 - 4\ell + 3}{2L^2} + O\left(\frac{\ell^3}{L^3}\right).$$

Substituting in (33) gives

$$\log M_r = N \left[L - \ell - 1 + \frac{\ell}{L} + \frac{(\ell - 1)^2}{2L^2} + O\left(\frac{\ell^3}{L^3}\right) \right] + O\left(\frac{L^2}{N}\right). \quad (35)$$

The remainder for w follows rigorously because substituting its displayed truncation into $w + \log w$ leaves a remainder of the stated order, and the derivative $1 + 1/w$ is bounded below by one. Formula (35) is a logarithmic expansion: dropping its subleading terms and claiming a relative asymptotic for the unlogged sequence would not be valid.

12 Rational generating functions and exact simulation

Once a recurrent core is known, every numerical or labeled leader generating function is rational. More generally, a finite selected projection in the admissible regime is eventually periodic. If its entry time is t_0 , its period is P , and token i occurs at the residue set $B_i \subseteq \{0, \dots, P - 1\}$ on that periodic part, then

$$\sum_{t \geq 0} \mathbf{1}_{\{L_t=i\}} z^t = \sum_{t < t_0} \mathbf{1}_{\{L_t=i\}} z^t + \frac{z^{t_0} \sum_{b \in B_i} z^b}{1 - z^P}. \quad (36)$$

The same formula, with coefficients equal to leader weights, gives the numerical leader series of an obstructed input. It need not hold for the entire infinite-alphabet word in the admissible alternative.

For (9), the numerical word has the generating function

$$2 + 100z + 2z^2 + 2z^3 + \frac{z^4 + 2z^5}{1 - z^2}. \quad (37)$$

The period-two tail here does not contradict the labeled period four.

12.1 A finite exact model from a prefix certificate

There is no need to materialize an infinite tail, or even a queue of length the largest throw. Given the obstruction certificate, retain only the positions of labels $0, \dots, J$, initially $(0, 1, \dots, J)$. A token from this set is always at zero, because Theorem 4.1 says no later label ever leads. A move changes these finitely many positions using its known weight. Each coordinate is bounded by $\max(i, x_i)$, so this is a finite exact state model of the *whole* leader word.

The function `certified_orbit` implements this model and detects its first repeated state. Its optional step cap is a resource limit; reaching it does not invalidate the static certificate or constitute a recurrence counterexample. Static core extraction and the formula for P can be fast even when explicit enumeration of a period is necessarily huge.

12.2 Avoiding a truncation error

A finite prefix is not automatically a closed queue. In particular, Python’s `list.insert(v, head)` silently appends if v exceeds the list length; using that operation on an insufficient prefix changes the process.

The function `finite_horizon` instead starts with at least $T + w$ input tokens to compute T moves and a final visible window of length w . A throw beyond the represented prefix is omitted rather than incorrectly appended. Such a token, and every unrepresented tail token, can move left by at most one per remaining step, so cannot reach the requested window in time. The representation loses at most one position per move and remains long enough for the promised horizon. This is an exact finite-horizon computation, not an empirical tail approximation.

13 Computational verification and reproducibility

All distributed Python code uses only the standard library. The following checks were actually executed; the raw counts, example words, and assertions are included in `data/`.

13.1 Every small closed input, with repeated weights distinguished

For each $2 \leq n \leq 7$, every ordered word in $\{1, \dots, n-1\}^n$ was simulated as a genuinely closed labeled queue until a state repeated. The check compared the recurrent labels and the labels ever visited with the static first-obstruction certificate, verified each individual rational frequency, checked the exact period, and confirmed that the tail was fixed on the eventual cycle.

n	Words checked	Critical words	Critical multisets	Max. period	Max. transient
2	1	1	1	2	0
3	8	4	2	4	2
4	81	27	5	9	6
5	1024	256	14	27	14
6	15625	3125	42	81	30
7	279936	46656	132	256	62

The total is 296 675 closed inputs. The transient column is an observed finite-range statistic, not a separately asserted sharp transient formula. The maximum-period column agrees with Theorem 11.1.

13.2 Complete bounded configuration spaces

For every admissible nondecreasing selected profile with weights at most seven and at most five tokens, all states in (16) were enumerated. The test verified the explicit inverse at every state, decomposed the entire permutation into cycles, and checked both periods and frequencies on each cycle.

Selected tokens	Profiles	States	Cycles	One-cycle profiles	Max. period
1	7	35	7	7	8
2	27	600	53	17	49
3	75	6500	358	19	216
4	165	48181	1865	25	625
5	297	247929	8025	11	1024

This covers 571 profiles, 303 245 bounded states, and 10 308 cycles. It tests the coprimality criterion against actual cycle decompositions, rather than merely comparing two algebraically equivalent formula implementations.

13.3 Additional independent checks

The waiting-or-barrier bound was checked in 5470 finite token-position cases. Sparse backward parking was compared with a sorted-prefix test on 1500 seeded random inputs, some containing weights around 10^{40} . The finite-horizon simulator was compared with sufficiently long closed reference queues on 200 cases. Separately, every critical stride profile through core size 13 was enumerated—290 511 profiles in total—and its period was compared with the sharp extremal formula. Finally, the escape product was bounded using exact fractions as in (26).

These are consistency checks over explicitly finite ranges. They are not Lean verification, exhaustive checks of all infinite inputs, or independent human review of the proofs.

13.4 Reproduction commands

From the archive’s top directory, run

```
python3 code/verify.py
python3 code/extra_results.py
pdflatex -interaction=nonstopmode -halt-on-error throwback_research.tex
pdflatex -interaction=nonstopmode -halt-on-error throwback_research.tex
```

The Python scripts write deterministic mathematical outputs, except for the recorded wall-clock runtime. Exact frequencies are stored as rational numbers or rational strings, not inferred from floating-point histograms.

14 Scope, remaining refinements, and conclusion

The repeated-entry question has a complete answer at the labeled level: a simple global counting condition decides universal recurrence, and the first bad prefix specifies the exact finite survivor set when it fails. Corollary 4.3 translates this to numerical values. The bounded-state analysis also yields a necessary and sufficient perfect-mixing criterion, rather than a collection of sufficient examples.

Several refinements are distinct from these resolved statements. With equal weights, collapsing token labels can reduce the period in ways that depend on the cycle. For example, selected weights (3, 3) have labeled projected period four, but their numerical projection can be 3, 3, *, * of period four or 3, *, 3, * of period two. Thus numerical period is not in general determined by the common

numerical frequencies alone. The article does not claim a complete enumeration of these quotient cycles. Nor does it claim a sharp formula for every capture transient, or a bit-optimal implementation for very large integer weights.

The recurrence and cycle proofs are elementary enough to suggest a compact formalization: finite positions, nested count inequalities, a finite permutation, and gcd arithmetic carry the main argument. No such formalization is included or claimed. The appropriate next validation step is independent checking of the proofs and a broader priority search before any submission or OEIS amendment.

The resulting picture is unusually explicit. A queue either keeps every occurrence active, or a finite prefix certificate predicts its permanent core. Finite projections consist of equal-length cycles controlled by simple clocks. Their exact periods identify when perfect mixing occurs, and the largest core period is a classical maximum-of-powers sequence with a precise Lambert- W scale. These deductions connect recurrence dynamics, parking functions, Catalan counting, rational generating functions, and asymptotic analysis in a single framework.

A A compact proof-audit checklist

The following points are the main places where an incorrect generalization could otherwise enter.

Occurrences are never identified. All positional arguments concern distinct labels, even when their weights coincide. Only after the recurrent labels are known are frequencies grouped by numerical value.

The obstruction is an initial-prefix obstruction. The smallest overloaded threshold of the whole input need not identify the actual core. Both the least prefix index J and then the least threshold inside that prefix are required.

Capture is proved in finite time. The argument does not assume that all globally small tokens eventually surface. Only tokens in the first bad prefix are forced to surface, using the admissibility of every earlier prefix and the waiting-or-barrier lemma.

The period is a least period. Periodicity alone would provide only an upper bound. The unique minimum symbol forces divisibility by its clock length, and the compressed survivor word forces the remaining divisibility condition. This proves minimality.

A frequency denominator alone need not give a period. The lcm formula is established by the clock recursion, not by the generally false inference that frequencies determine every periodic word. Numerical quotients explicitly demonstrate that distinction.

Infinite density is handled by finite telescoping first. Equation (23) is obtained from finite sums before passing to a monotone product limit. No unjustified exchange of a countable sum and a time-average limit is used.

Enumeration and numerical experiments have separate roles. The Catalan and power counts are proved by cyclic minima. Enumerations check those proofs on finite ranges, and do not stand in for an all-size argument.

B Code and data inventory

File	Purpose
<code>code/throwback.py</code>	Sparse static classifier, independent certificate verifier, bounded map and inverse, exact rational statistics, safe finite horizon, and certified prefix orbit.
<code>code/verify.py</code>	Exhaustive closed queues and bounded configuration spaces, plus waiting, sparse, and horizon checks.
<code>code/extra_results.py</code>	Exhaustive critical profiles through size 13, worked examples, and rational escape-product bounds.
<code>code/analyze.py</code>	Command-line interface for prefix certificates and exact orbit analysis.
<code>data/verification.json</code>	Executed verification ranges and exact totals.
<code>data/finite_queue_checks.csv</code>	Closed-input enumeration table.
<code>data/bounded_configuration_checks.csv</code>	Full finite permutation checks.
<code>data/extremal_periods.csv</code>	Maximum periods and attaining profiles.
<code>data/examples.json</code>	Exact transient and periodic example words.
<code>data/escape_product_bounds.json</code>	Certified rational-product interval, displayed with high-precision decimal endpoints.
<code>notes/source_status.md</code>	Sources consulted and the scope of the novelty/status check.

References

- [1] J. Darnell and B. F. Dribus, *Throwback Sequences of Positive Integers*, Journal of Integer Sequences **28** (2025), Article 25.5.1, 18 pp. <https://cs.uwaterloo.ca/journals/JIS/VOL28/Dribus/dribus4.pdf>.
- [2] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A357081, “Leader at step n of the THROWBACK procedure.” <https://oeis.org/A357081>. Accessed 19 September 2026.
- [3] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A000108, Catalan numbers. <https://oeis.org/A000108>. Accessed 19 September 2026.
- [4] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A000312, $a(n) = n^n$, including prime parking functions of length $n + 1$. <https://oeis.org/A000312>. Accessed 19 September 2026.
- [5] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A003320, $a(n) = \max_{0 \leq k \leq n} k^{n-k}$. <https://oeis.org/A003320>. Accessed 19 September 2026.